



Cloning

▼ Type	 Lecture
📅 Date	@February 13, 2022
☰ Lecture #	1
🔗 Lecture URL	https://youtu.be/aJxPvy1poaU
🔗 Notion URL	https://21f1003586.notion.site/Cloning-9242921d53284a55a78a90c339dd8230
# Week #	8

Copying an object

- Normal assignment creates two references of the same object
 - Basically, two different variables point to the same object in memory
 - Updating via any name will update the same object
- But, what if we want two separate but identical objects?

```
public class Employee {  
    private String name;  
    private double salary;  
}
```

```

    public Employee(String name, String salary) {
        this.name = name;
        this.salary = salary;
    }

    public void setName(String name) {
        this.name = name;
    }
}

...
Employee e1 = new Employee("Dhruv", 21500.0);
Employee e2 = e1;
e2.setName("Eknath");    // This updates e1 as well

```

- `e2` should be initialized to a disjoint copy of `e1`
- How does one make a faithful copy?

The `clone()` method

- `Object` defines a method `clone()`
- `e1.clone()` returns a bitwise copy of `e1`
- Why a bitwise copy?
 - `Object` does not have access to the private instance variables
 - So, it cannot build up a fresh copy of `e1` from scratch
- What could go wrong with a bitwise copy?

```

public class Employee {
    private String name;
    private double salary;

    public Employee(String name, String salary) {
        this.name = name;
        this.salary = salary;
    }

    public void setName(String name) {
        this.name = name;
    }
}

...
Employee e1 = new Employee("Dhruv", 21500.0);
Employee e2 = e1.clone();
e2.setName("Eknath");    // This does NOT update e1

```

Shallow copy

- What if we add an instance variable `Date` to `Employee` ?
 - Assume `update()` updates the components of a `Date` object

```
public class Employee {
    private String name;
    private double salary;
    private Date birthday;
    ...
    public void setName(String name) {
        this.name = name;
    }

    public void setbday(int dd, int mm, int yy) {
        birthday.update(dd,mm,yy);
    }
}

...
Employee e1 = new Employee("Dhruv", 21500.0);
Employee e2 = e1.clone();
e2.setName("Eknath");    // This does NOT update e1.name
e2.setbday(16,4,1996);   // This CHANGES e1.bday
```

- Bitwise copy made by `e1.clone()` copies the reference to the embedded `Date`
 - `e2.birthday` and `e1.birthday` refer to the same object
 - `e2.setbday()` affects `e2.birthday`
- Bitwise copy is a **shallow copy**
 - Nested mutable reference are copied verbatim

Deep copy

- Deep copy recursively clones the nested objects

```
public class Employee {
    private String name;
    private double salary;
    private Date birthday;
    ...
    public void setName(String name) { ... }

    public void setbday(...) { ... }

    public Employee clone() {
        Employee newEmp = (Employee) super.clone();
        Date newBday = birthday.clone();
    }
}
```

```

        newEmp.birthday = newBday;
        return newEmp;
    }
}

```

- Override the shallow `clone()` from `Object`
- `Object.clone()` returns an `Object`
 - Cast `super.clone()`
- `Employee.clone()` returns an `Employee`
 - Allowed to change the return type

Deep copy ...

- What if `Manager` extends `Employee`?
- New instance variable `promodate`

```

public class Employee {
    private String name;
    private double salary;
    private Date birthday;
    ...
    public void setName(String name) { ... }

    public void setbday(...) { ... }

    public Employee clone() { ... }
}

public class Manager extends Employee {
    private Date promodate;
    ...
}

```

- Manager inherits deep copy `clone()` from `Employee`
- However, `Employee.clone()` does know that it has to deep copy `promodate`
- Cloning is subtle, so Java puts in some restrictions

Restrictions on `clone()`

- To allow `clone()` to be used, a class has to implement `Cloneable` interface
 - **Marker interface**

```

public class Employee implements Cloneable {
    private String name;
    private double salary;

    public void setName(String name) { ... }

    public void setbday(...) { ... }
}

...
Employee e1 = new Employee("Dhruv", 21500.0);
Employee e1 = e1.clone();
e2.setName("Eknath");    // This does NOT update e1

```

- `clone()` in `Object` is `protected`
 - Only `Employee` objects can `clone()`
- Re-define `clone()` as `public` to allow other classes to clone `Employee`
 - Expanding visibility from `protected` to `public` is allowed
- `Object.clone()` throws `CloneNotSupportedException`
 - Catch or report this exception
 - Call `clone()` in `try` block

```

public class Employee implements Cloneable {
    private String name;
    private double salary;
    private Date birthday;
    ...
    public void setName(String name) { ... }

    public void setbday(...) { ... }

    public Employee clone() throws CloneNotSupportedException { ... }
}

```

Summary

- Making a faithful copy of an object is a tricky problem
- Java provides a `clone()` function in `Object` that does shallow copy
 - However, shallow copy aliases nested objects
- Deep copy solves this problem, but inheritance can create complications

- To force programmers to consciously think about these subtleties, Java puts in some checks to using `clone()`
- Must implement marker interface `Cloneable` to allow `clone()`
- `clone()` is `protected` by default, override as `public` is needed
- `clone()` in `Object` throws `CloneNotSupportedException`, which must be taken into account when overriding



Type Inference

▼ Type	 Lecture
📅 Date	@February 13, 2022
☰ Lecture #	2
🔗 Lecture URL	https://youtu.be/o1cQO3iB2Ok
🔗 Notion URL	https://21f1003586.notion.site/Type-Inference-8c7d6d6f90c543498b68c1486ebb73d3
# Week #	8

Type declarations vs type inference

- Java insists that all variables are declared in advance, with type information
- The compiler can then check whether the program is well-typed

```
public class Employee { ... }  
  
public class Manager extends Employee { ... }  
  
Employee e;
```

```

Manager m;

...

m = new Manager(...);
e = m; // Allowed by sub-typing

```

- An alternative approach is to do type inference
- Derive type information from context

- For instance, `s` should be a `String`

```
s = "Hello, " + "world";
```

- Propagate type information: now `t` is also `String`

```
t = s + 5;
```

Type inference

- Assume the code is well-typed, derive most general types
- Use information from constants to determine type

```
s = "Hello, " + "world";
```

- Propagate type information based on already inferred types

```
t = s + 5;
```

```

public class Employee { ... }
public class Manager extends Employee {
    ...
    public double bonus(...) { ... }
}
...
public static f(Employee x) {
    ...
    double d = x.bonus(...);
    // x must be a Manager?
    ...
}

```

- If `x.bonus()` is legal, `x` must be `Manager` rather than `Employee`
- Keep track of and validate type obligations

Type inference

- Assume program is type-safe, derive most general types compatible with code
- Use information from constants to determine type

- Propagate type information based on already inferred types
- Typing judgements should ideally be made at compile time, not at run-time
 - Static analysis of code
- Balance flexibility with algorithmic tractability

Type inference in Java

- Java allows limited type inference
 - Only for local variables in a function
 - Not for instance variables of a class
- Use generic `var` to declare variables
 - Must be initialized when declared
 - Type is inferred from initial value
- Be careful about format for numeric constants
- For classes, infer most constrained type
 - `e` is inferred to be `Manager`
 - `Manager` extends `Employee`
 - If `e` should be `Employee`, declare explicitly

```
var b = false; // boolean
var s = "Hello, world!"; // string
var d = 2.0; // double
var f = 3.14f; // float
var e = new Manager(...); // Manager
```

Summary

- Automatic type inference can avoid redundancy in declarations


```
Manager m = new Manager(...);
```
- Assuming the program is type-safe, derive most general types compatible with the code
 - Compiler can infer type from expressions used to assign values
 - Inferred type information can be propagated

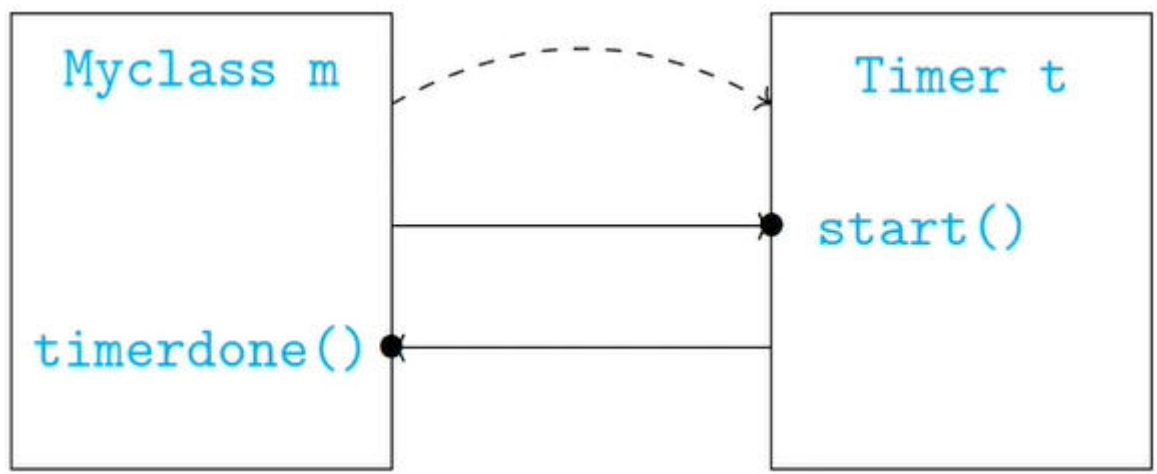
- Challenge is to do this statically, at compile-time
- Java allows limited type inference
 - Only local variables that are initialized when they are declared



Higher Order Functions

▼ Type	 Lecture
📅 Date	@February 13, 2022
☰ Lecture #	3
🔗 Lecture URL	https://youtu.be/4nrbgx0e4RA
🔗 Notion URL	https://21f1003586.notion.site/Higher-Order-Functions-ee9b794297e943719bb1f9f0fca7e2f9
# Week #	8

Passing Functions



- Recall callbacks
 - `Myclass m` creates a `Timer t`
 - `t` starts running in parallel
 - `t` notifies `m` when the time limit expires
- `m` needs to pass `timerdone()` to `t`
- Achieved this through an interface

```

public interface Timerowner {
    public abstract void timerdone();
}

public class Myclass extends Timerowner {
    ...
}
  
```

```

public class Timer implements Runnable {
    private Timerowner owner;
    ...
    public void start() {
        ...
        owner.timerdone();
    }
}
  
```

Passing functions

- Customize `Arrays.sort`
- `Comparator` interface provides signature for comparison function

- Implement `Comparator`
- Pass to `Arrays.sort`

```
public interface Comparator<T> {
    public abstract int compare(T o1, T o2);
}

public class StringCompare implements Comparable<String> {
    public int compare(String s1, String s2) {
        return s1.length() - s2.length();
    }
}

String[] strarr = new ...;
Arrays.sort(strarr, StringCompare);
```

Functional interfaces

- Interfaces that define a single functions are called functional interfaces
 - `Comparator, TimerOwner`
- Is there a way to pass the required functions?

```
public interface Comparator<T> {
    public abstract int compare(T o1, T o2);
}

public interface Timerowner {
    public abstract void timerdone();
}
```

- In Python, function names are similar to variable names
 - Define a function
 - Pass it as an argument to another function
 - `map` is a higher order function

```
def square(x):
    return x*x

l = list(map(square, range(100)))
```

Lambda expressions

- Lambda expressions denote anonymous functions
 - `(Parameters) -> Body`
 - Return value and type are implicit
- From λ -calculus (Alonzo Church)
 - Foundational model for computing, parallel to Alan Turing's machines
 - Basis for functional programming: Lisp, Scheme, ML, Haskell
- Substitute wherever a functional interface is specified
- Limited type inference is also possible
 - Java infers `s1` and `s2` are `String`

```
(String s1, String s2) -> s1.length() - s2.length()

String[] strarr = new ...;
Arrays.sort(strarr, (String s1, String s2) -> s1.length() - s2.length());

String[] strarr = new ...;
Arrays.sort(strarr, (s1, s2) -> s1.length() - s2.length());
```

- More complicated function body can be defined as a block
- Note that the function is anonymous only for the caller
- The function that receives the lambda expression still needs to use a functional interface for the parameter type

```
public static<T> void
    Arrays.sort(T[] a, Comparator<T> c)}
```

- Inside `Arrays.sort()`, refer to the function by the name `compare()` defined in the `Comparator` interface

```
(String s1, String s2) -> {
    if s1.length() < s2.length()
        return -1;
    else if s1.length() > s2.length()
        return 1;
    else
        return 0;
}
```

Passed named functions

- If the lambda expression consists of a single function call, we can pass that function by name
 - Method reference
- We saw an example with adding entries to a `Map` object
 - Here `sum` is a static method in `Integer`

```
map<String, Integer> scores = ...;  
scores.merge(bat, newscore, Integer::sum);
```

- Here's the corresponding expression, assuming type inference

```
(i,j) -> Integer::sum(i,j)
```

- Expression should call a function and nothing else — this expression cannot be replaced by a method reference

```
(i,j) -> Integer::sum(i,j) > 0
```

Method references

- `ClassName::StaticMethod`
 - Method reference is `C::f`
 - Corresponding expression with as many arguments as `f` has

```
(x1, x2, ..., xk) -> f(x1, x2, ..., xk)
```

- `ClassName::InstanceMethod`
 - Method reference is `C::f`
 - Called with respect to an object that becomes implicit parameter

```
(o, x1, x2, ..., xk) -> o.f(x1, x2, ..., xk)
```

- `object::InstanceMethod`
 - Method reference is `o::f`
 - Arguments are passed to `o.f`

```
(x1, x2, ..., xk) -> o.f(x1, x2, ..., xk)
```

- Can also pass references to constructors

Summary

- Many languages support higher-order functions
 - Passing a function as an argument to another function
- In object-oriented programming, this is achieved using interfaces
 - Encapsulate the function to be passed as an object
- Java allows functions to be passed directly in place of functional interfaces
 - Interface consists of a single function
- Lambda expressions describe anonymous functions
 - Cannot pass lambda expressions in general
 - Only when the argument is a functional interface
- Can pass a method reference if the lambda expression consists of a single function call



Streams

▼ Type	 Lecture
📅 Date	@February 13, 2022
☰ Lecture #	4
🔗 Lecture URL	https://youtu.be/xs94mCRwqVQ
🔗 Notion URL	https://21f1003586.notion.site/Streams-da5ae640124a45ca9892e8584a5e1cdd
# Week #	8

Operating on collections

- We usually use an iterator to process a collection
 - Suppose we have split a text file as a list of words
 - We want to count the number of long words in the list
- An iterator generates all elements from a collection as a sequence
- Alternative approach
 - Generate a stream of values from a collection

- Operations transform input streams to output streams
- Terminate with a result

```
List<String> words = ...;
long count = 0;
for(String w : words) {
    if(w.length() > 10) {
        count++;
    }
}

long count = words.stream().filter(w -> w.length() > 10).count();
```

Why streams?

- Stream processing is declarative
 - Focus on what to compute, rather than how to
- Processing can be parallelized
 - `filter()` and `count()` in parallel
- Lazy evaluation is possible
 - Suppose we want first 10 long words
 - Stop generating the stream once we find 10 such words
 - Need not generate the entire stream in advance
 - Can even work, in principle, with infinite streams

```
long count = words.stream().filter(w -> w.length() > 10).count();
```

```
long count = words.parallelStream().filter(w -> w.length() > 10).count();
```

Working with streams

- Create a stream
- Pass through intermediate operations that transform streams
- Apply a terminal operation to get a result
- A stream does not store its elements

- Elements stored in an underlying collection
- Or generated by a function, on demand
- Stream operations are non-destructive
 - Input stream is untouched

Creating Streams

- Apply `stream()` to a collection
 - Part of `Collection` interface
- Use static method `Stream.of()` for arrays
- Static method `Stream.generate()` generates a stream from a function
 - Provide a function that produces values on demand, with no argument
- `Stream.iterate()` — a stream of dependent values
 - Initial value, function to generate the next value from the previous one
 - Terminate using a predicate

```
List<String> wordlist = ...;
Stream<String> wordstream = wordlist.stream();

String[] wordarr = ...;
Stream<String> wordstream = Stream.of(wordarr);

Stream<String> echos = Stream.generate(() -> "Echo");

Stream<Double> randomds = Stream.generate(Math::random);

Stream<Integer> integers = Stream.iterate(0, n -> n + 1);

Stream<Integer> integers = Stream.iterate(0, n -> n < 100, n -> n + 1);
```

Processing streams

- `filter()` to select elements
 - Takes a predicate as arguments
 - Filter out the long words

```
List<String> wordlist = ...;
Stream<String> longwords = wordlist.stream().filter(w -> w.length() > 10);
```

- `map()` applies a function to each element in the stream
 - Extract the first letter of each long word

```
List<String> wordlist = ...;
Stream<String> startlongwords =
    wordlist.stream()
        .filter(w -> w.length() > 10)
        .map(s -> s.substring(0,1));
```

- What if `map()` function generates a list?
 - Suppose we have `explode(s)` that returns the list of letters in `s`
 - `map()` produces stream with nested lists

```
List<String> wordlist = ...;
Stream<String> startlongwords =
    wordlist.stream()
        .filter(w -> w.length() > 10)
        .map(s -> explode(s));
```

- `flatMap()` flattens (collapses) nested list into a single stream

```
List<String> wordlist = ...;
Stream<String> startlongwords =
    wordlist.stream()
        .filter(w -> w.length() > 10)
        .flatMap(s -> explode(s));
```

Stream transformations

- Make a stream finite — `limit(n)`
 - Generate 100 random numbers

```
Stream<Double> randomds =
    Stream.generate(Math::random).limit(100);
```

- Skip `n` elements — `skip(n)`
 - Discard the first 10 random numbers

```
Stream<Double> randomds =  
  Stream.generate(Math::random).skip(10);
```

- Stop when element matches a criterion — `takeWhile()`
 - Stop with number smaller than 0.5

```
Stream<Double> randomds =  
  Stream.generate(Math::random)  
  .takeWhile(n -> n >= 0.5);
```

- Stop when element matches a criterion — `takeWhile()`
 - Stop with number smaller than 0.5

```
Stream<Double> randomds =  
  Stream.generate(Math::random)  
  .takeWhile(n -> n >= 0.5);
```

- Start after elements matches a criterion — `dropWhile()`
 - Start after number larger than 0.05

```
Stream<Double> randomds =  
  Stream.generate(Math::random)  
  .dropWhile(n -> n <= 0.05);
```

- Can also combine streams, extract distinct elements, sort, ...

Reducing a stream to a result

- Number of elements — `count()`
 - Count random numbers larger than 0.1
- Largest and smallest values seen
 - `max()` and `min()`
 - Requires a comparison function
 - What happens if the stream is empty?
 - Return value is option type — later

- First element — `findFirst()`
 - First random number above 0.999
 - Again, deal with empty stream

```
long countrand =
  Stream.generate(Math::random)
    .limit(100)
    .filter(n -> n > 0.1)
    .count();
```

```
Optional<Double> maxrand =
  Stream.generate(Math::random)
    .limit(100)
    .filter(n -> n < 0.001)
    .max(Double::compareTo);
```

```
Optional<Double> firstrand =
  Stream.generate(Math::random)
    .limit(100)
    .filter(n -> n > 0.999)
    .findFirst();
```

Streams

- We can view a collection as a stream of elements
- Process the stream rather than use an iterator
- Declarative way of computing over collections — popular in functional programming
- Create a stream, transform it reduce it to a result
- Can create a stream from any collection, or generate from a function
- Stream transformations are non-destructive: filter, map, limit to a finite number, skip elements, ...
- Various functions to reduce to a result — deal with empty streams